QA TESTING GUIDE

AI-First SDLC Platform — Set up, understand, and test the whole application end to end

A complete, plain-language handbook for the QA team. It takes you from an empty laptop to running the platform, understanding every feature, and testing each user journey from sign-up to deployment — so you can find and log issues on your own, without waiting on the development team.

Prepared for	The QA Testing Team — EdvenSoft
What you will be able to do	Install everything, create your own workspace, connect your machine, trigger every phase, and log bugs in a shared, review-ready format.
Assumed knowledge	Comfortable using web apps and a terminal to copy-paste commands. No coding or platform-internals knowledge required — every term is explained.
Accuracy	Audited directly against the live source code on the date below. Where an older internal doc disagrees, THIS guide is correct.
Date	September 15, 2026

Contents

1. How to Use This Guide 4

1.1 Why you have this guide . . . 4

1.2 What you are testing, in one line . . . 4

1.3 How to read it . . . 4

2. What the SDLC Platform Is 4

2.1 The big idea . . . 4

2.2 Who uses it and what makes it different . . . 5

2.3 The seven house rules (why the app behaves the way it does) . . . 5

3. What the Application Does for a User 5

3.1 The capabilities at a glance . . . 6

3.2 The shape of a normal working session . . . 6

4. Glossary — Every Word You Will See 6

5. Setting Up Your Machine (one time) 7

5.1 What to install (with versions) . . . 7

5.2 Step 1 — Get the code . . . 8

5.3 Step 2 — Log the Claude Code CLI into your own subscription . . . 8

5.4 Step 3 — Create your .env settings file . . . 8

5.5 Step 4 — Start the application . . . 9

5.6 Step 5 — Create your own workspace . . . 9

5.7 Step 6 — Connect your machine (the companion) . . . 9

6. The Features You Will Test (screen by screen) 10

6.1 Signing in and onboarding . . . 10

6.2 Setting up a project . . . 10

6.3 Connecting real tools (Integrations) . . . 11

6.4 Work Items . . . 11

6.5 Phases & Runs . . . 12

6.6 Clarifications and Gates (staying in control) . . . 12

6.7 Seeing everything (observability) . . . 12

6.8 Brownfield onboarding . . . 12

6.9 Admin (workspace admins only) . . . 12

7. The Phase Catalog (the 16 lifecycle steps) 13

7.1 The three kinds of phase, in plain terms . . . 13

7.2 The order you would run them . . . 14

8. The Companion, in Detail 14

8.1 What it is and why it exists . . . 14

8.2 The three commands . . . 14

8.3 The serve options that matter . . . 14

8.4 Which setup for which testing . . . 15

9. End-to-End User Journeys (walk-throughs) 15

9.1 Journey A — Greenfield, single repo (the golden path / smoke test) . . . 15

9.2 Journey B — A build phase (code actually changes) . . . 16

9.3 Journey C — The security scan (read-only) . . . 16

9.4 Journey D — Brownfield onboarding . . . 17

9.5 Journey E — Clarifications and offline takeover (cross-cutting) . . . 17

10. Is It a Bug, or Is It Meant to Work That Way? 17

11. Testing in Parallel Without Stepping on Each Other 18

12. Troubleshooting (fix it yourself first) 19

13. How to Report a Bug (the format that gets it fixed) 19

13.1 Keep your own bug sheet . . . 19

13.2 Priority — how bad is it? . . . 20

13.3 Attaching screenshots (important gotcha) . . . 20

13.4 What makes a report actionable . . . 20

13.5 A worked example . . . 21

1. How to Use This Guide

Read this page first. It tells you what this document is for, how the QA team fits into the project, and the one rule that makes your testing useful to the developers.

1.1 Why you have this guide

This guide is written so you can do everything on your own: install the tools, create your own account, run the full application, and log the problems you find. You should only need to interrupt a developer for a genuinely critical blocker (and Section 12 lists the few things only they can hand you).

1.2 What you are testing, in one line

The AI-First SDLC Platform is a web application where AI agents run a software project's whole lifecycle — from the first idea, through requirements, design, coding, testing, and security, to deployment — with a human approving every important step. Your job is to walk through all of that as a real user would and report anything that looks wrong, confusing, broken, or ugly.

1.3 How to read it

- **Sections 2-4** explain *what* the platform is and define every word you will see on screen. Read these once to build a mental model.
- **Section 5** is the one-time **setup** of your laptop. Follow it top to bottom.
- **Sections 6-8** are the **reference**: every feature, every phase, and the exact end-to-end journeys. Keep these open while you test.
- **Section 9** is your **first smoke test** — a short golden path to confirm your setup works before you explore freely.
- **Sections 10-13** tell you what is *normal* vs. a *bug*, how to avoid stepping on each other, how to fix common setup snags yourself, and how to log a bug so the developers can act on it immediately.

> **The one rule that makes your bugs useful**
>
> Before you log something as a bug, check Section 10 (“Is it a bug or is it meant to work that way?”). Some behaviours look wrong but are deliberate design decisions (for example, a run can stop because of an AI usage limit — that is not an application bug). Logging those as defects wastes everyone's time.

2. What the SDLC Platform Is

Before you can test something, you need a clear picture of what it is supposed to be. This section gives you that picture in everyday language.

2.1 The big idea

“SDLC” stands for **Software Development Life Cycle** — the full journey a piece of software takes: somebody has an idea, it becomes written requirements, then a product spec, then an architecture and a design, then code, then it gets reviewed, tested, security-checked, and finally shipped. Traditionally a whole team of specialists does each step by hand.

This platform does that journey with a **team of AI agents** instead — one specialist agent per step (an "ideation" agent, a "requirements" agent, an "architecture" agent, and so on). But it is **not** "an AI that writes code and hopes for the best." A human reviews and approves the output of every step before the project moves on. The platform's job is to keep all of this organised, safe, traceable, and repeatable.

2.2 Who uses it and what makes it different

- It is **multi-tenant**: many separate companies ("tenants") can use the same platform, and no tenant can ever see another's data. When you sign up, you create your *own* tenant — your own private workspace.
- It is **web-delivered**: the whole thing runs in a browser. There is one small helper program (the "companion") that runs on your own laptop — more on that in Section 4.
- It works with a team's **real tools**: instead of inventing its own copies of your Jira, Confluence, Figma, GitHub, or Xray, it connects to the real ones and reads/writes them — but only after a human approves.
- It is **engine-agnostic**: the AI "brain" behind the agents can be swapped. Today it uses Claude; the platform is built so it could use another AI later without rewriting the agents.

2.3 The seven house rules (why the app behaves the way it does)

The whole system obeys these locked principles. Knowing them helps you tell a real bug from intended behaviour.

#	Rule	What it means for you as a tester
1	State vs. content	The platform stores **pointers and history** (who did what, when, which version). The actual documents live in the connected tools or a file store. So a "PRD" you see is a tracked pointer to a body of text, not a random file.
2	Orchestration vs. execution	The platform decides *what* runs and *when*. The AI only *does* the step. Approvals ("gates") are always the platform's job, never the AI's.
3	Memory = index, not copies	It remembers **versions and fingerprints** of documents, not duplicate copies. This is how it can later flag "this changed."
4	Work is a graph	The unit of work is a **Work Item**. Items can depend on each other; they do not have to run in one rigid line.
5	Every output is gated	Draft -> reviewed -> approved -> published. Nothing is auto-published. A human always approves before anything reaches a real tool.
6	Strict tenant isolation	Every record is tagged with its tenant and protected by the database. You will never see another workspace's data (if you do, that is a serious bug — log it P1).
7	Capabilities, not vendors	The app thinks in **capabilities** ("task tracker", "doc store") and binds a vendor (Jira, Confluence) underneath. That is why the Integrations screen is organised by capability.

3. What the Application Does for a User

This section is the "menu" of what a user can actually accomplish. Each item here becomes something you will test in detail later.

3.1 The capabilities at a glance

- **Create a workspace and invite a team** — sign up, become the workspace admin, invite teammates, and assign each person roles (product owner, architect, QA lead, etc.).
- **Set up a project** — choose whether it is **greenfield** (brand new) or **brownfield** (existing code and docs), and whether it lives in **one repo** or **many repos**.
- **Connect your real tools** — attach Confluence, Jira, Figma, GitHub, and Xray so the platform can read and (after approval) write to them.
- **Connect your machine** — run the small “companion” so AI phases execute on your computer using your own Claude subscription.
- **Run the lifecycle** — trigger each phase (ideation, requirements, PRD, architecture, UX, task breakdown, implementation, testing, test automation, security, CI/CD, deployment) on a work item.
- **Stay in control** — answer the AI's clarifying questions, review its drafts, and approve or reject at each gate.
- **See everything** — a dashboard of what needs you, a full history of runs and transcripts, an artifact registry, and a traceability view linking requirements to code to tests.
- **Onboard existing projects** — for brownfield, ingest existing documents, index the code, and get a “gap report” of what is missing.

3.2 The shape of a normal working session

Most of what you will test follows this rhythm, over and over:

1. Pick or create a **work item** (a feature, story, or task).
2. **Trigger a phase** on it (say, “requirements”).
3. The AI may pause and **ask you clarifying questions** — you answer them.
4. The AI produces a **draft artifact** (a document, a plan, code, etc.).
5. A **gate** opens: the right role reviews the draft and **approves or rejects** it.
6. On approval, the work item advances and (if you chose to) the result can be **published** to a real tool like Confluence or Jira.

> **What you are really testing**
>
> You are testing whether this rhythm is smooth, clear, and correct on every screen: do buttons work, is the wording clear, does the right thing happen, are errors handled gracefully, and does the UI look right? You do **not** need to judge whether the AI's writing is “good” — focus on the application around it.

4. Glossary — Every Word You Will See

The screens use a handful of special terms. Learn these ten and nothing in the app will feel foreign. Keep this page bookmarked.

Term	Plain-language meaning
Workspace / Tenant	Your private company account on the platform. You create one at sign-up and become its admin. Everything you make lives inside it; no other workspace can see it.
Project	A single software project inside your workspace. It is either **greenfield** (new) or **brownfield** (existing), and uses one or many repos.
Work Item	The unit of work. Types you will see: **Epic** (big), **Story** (default), **Task**, **Defect** (a bug the platform raised), and **Test Case**. Each shows a friendly key like WI-14, never a raw ID.
Phase	One step of the lifecycle done by one specialist AI agent — e.g. “requirements”, “architecture”, “security”. There are 16 of them (Section 7).
Run	One execution of a phase on a work item. It has a live console and, afterwards, a full transcript and a cost record.
Artifact	A thing a phase produces — a PRD, an architecture doc, a review, test cases, a security report. The platform tracks its version and fingerprint.
Gate (Approval)	The human checkpoint after a phase. The right **role** reviews the draft, ticks any required criteria, and approves or rejects. Nothing moves on without it.
Clarification	A question the AI asks you mid-run when it needs a decision. While it waits, nothing is being generated, so it costs nothing. You answer and the run resumes.
Companion (Runner)	A small program you run on your own laptop. It lets AI phases execute on your machine using **your own Claude subscription** instead of a shared bill. The app shows “Companion: Ready” when it is running.
Integration / Capability	A connection to a real external tool. The app groups them by **capability** (doc store, task tracker, code host, design source, test management, code intelligence) and binds a vendor (Confluence, Jira, GitHub, Figma, Xray) underneath. Code intelligence is built in and set up automatically, so it has no card to connect.
System of Record (SoR)	The real external tool that officially owns a piece of content — e.g. Jira owns issues, Confluence owns pages. The platform publishes to the SoR only after a gate is approved.
Greenfield / Brownfield	**Greenfield** = a brand-new project starting from an idea. **Brownfield** = an existing project with code and docs the platform must first ingest and understand.

5. Setting Up Your Machine (one time)

Do this once, carefully, top to bottom. At the end you will have the whole application running on your laptop and connected to the team's shared cloud. Budget about an hour the first time.

> **The setup model, in one picture**
>
> You do **not** install a database or the cloud services. Those live once in the team's shared AWS (Postgres + Temporal + S3). You only run the **app** on your laptop (backend, worker, frontend) pointed at that shared cloud, plus the **companion**. This is exactly how the two developers already work — you are joining the same shared brain.

5.1 What to install (with versions)

Install these five things first. The versions are what the project is built and tested against — using much older versions is the most common cause of setup pain.

Tool	Version	Why you need it	Where to get it
Docker Desktop	Latest stable (4.x)	Runs the app services (backend, worker, frontend) in containers so you don't hand-install Python/Node for the app.	docker.com/products/docker-desktop
Git	Latest 2.x	To clone the project code and for the companion to work with repos.	git-scm.com
Python	3.12.x	To run the **companion** on your laptop.	python.org/downloads
Node.js	20 LTS or newer	Needed to install the Claude Code CLI (below).	nodejs.org
Claude Code CLI	Latest	The AI engine the companion drives. You log into it with your own Claude subscription.	npm i -g @anthropic-ai/claude-code

The two things only the dev team can give you

Almost everything here you do yourself. The exception is a handful of **shared secrets** you cannot invent: (1) the shared database connection string and password, the shared Temporal address, and the shared file-store (S3) keys and bucket — these go into your .env file in Step 3; and (2) confirmation of your own Claude subscription login. Ask a developer for the filled-in .env values once; everything else in this guide you can do without them.

5.2 Step 1 — Get the code

Open a terminal, go to the folder where you keep projects, and clone the repository (ask a developer for the exact repository URL if you don't have it):

```
git clone <the-repository-url> AI-First-SDLC-App
cd AI-First-SDLC-App
```

5.3 Step 2 — Log the Claude Code CLI into your own subscription

The companion runs AI phases using **your** Claude login, never a shared API key (the companion actually refuses to start if it detects an API key). Log in once:

```
claude       # then follow the prompt to sign in with your Claude account
```

Verify it worked by typing claude again — it should open without asking you to log in. Keep this account handy; each tester uses their own.

5.4 Step 3 — Create your .env settings file

The project reads all its settings from a single file called .env in the project root. Copy the template and then fill in the shared values from the dev team:

```
cp .env.example .env      # (Windows PowerShell: copy .env.example .env)
```

Open .env in a text editor and set these to the **shared cloud** values the dev team gives you (leave the rest at their defaults for now):

Setting in .env	What to put
DATABASE_URL	The shared AWS Postgres connection string (the sdlc_app role — the everyday, non-admin one). Example shape: postgres://sdlc_app:<password>@<aws-host>:5432/sdlc_platform
TEMPORAL_HOST	The shared Temporal address, e.g. <aws-host>:7233.
S3_ENDPOINT_URL / S3_BUCKET / S3_ACCESS_KEY / S3_SECRET_KEY / S3_REGION	The shared file-store values. For real AWS S3, S3_ENDPOINT_URL is left empty.
CORS_ALLOWED_ORIGINS	Leave as http://localhost:5173 (that is your local frontend).
ANTHROPIC_API_KEY	You can leave the placeholder. Phases run through your companion (your Claude login), so a key is not needed for normal testing.

> **Keep .env private**
>
> Your .env holds real shared passwords. It is already excluded from Git — never paste its contents into a chat, screenshot, or bug report.

5.5 Step 4 — Start the application

You do **not** start a local database. You start only the three app services and tell Docker *not* to spin up its own local infra (that is what --no-deps does), so everything points at the shared cloud:

```
docker compose --profile app up --no-deps backend worker frontend
```

The first run downloads and builds images, so give it a few minutes. When it settles, open your browser to:

```
http://localhost:5173
```

You should see the platform's **Log in** screen. If you do, your app is running. If not, jump to Section 12 (Troubleshooting) — it is almost always the .env values or Docker not being started.

5.6 Step 5 — Create your own workspace

You are a brand-new user, so you make your own account (you do **not** reuse the developers' data):

1. On the Log in screen, click **“New to Edvensoft? Create a workspace.”**
2. On **Sign up**, enter your **Full name**, **Work email**, a **Password**, and a **Workspace / org name** (e.g. “QA Sandbox — Priya”). Click **Create workspace**.
3. You become the **admin** of your new workspace. Click **Continue** to reach the **Setup checklist**.

> **Give yourself a clearly-named workspace**
>
> Since several people share the same cloud, name your workspace so it is obviously yours (your name + “QA”). This makes it easy to tell your test data apart and is important for testing in parallel (Section 11).

5.7 Step 6 — Connect your machine (the companion)

The last setup piece lets AI phases actually run. The app walks you through it:

1. From the Setup checklist (or the sidebar) open **Connect Machine**.
2. Click **Generate enrollment token** and copy the one-time token it shows (it is shown only once).
3. The screen shows you the exact command to run, already filled in with your workspace and token. Open a **second terminal**, go to the project folder, and run it (shape below).

The command looks like this — the app fills in the real values for you:

```
python -m runner.companion serve \
  --api-url http://localhost:8000 \
  --tenant <your-workspace-id> \
  --token <the-enrollment-token> \
  --worktrees C:\path\to\sdlc-worktrees \
  --repo-path C:\path\to\a-cloned-repo
```

Leave that terminal running. Back in the browser, the **Companion** pill in the top bar should switch to **"Ready"**. You are now fully set up. Section 8 explains the companion's options in detail (single-repo vs multi-repo, and when the --worktrees / --repo flags matter).

> **First-time companion notes**
>
> The companion needs the claude CLI logged in (Step 2) or it refuses to start with a clear "CANNOT START" message. For build and security phases it also needs a real cloned repo and the --worktrees folder — for pure planning phases (ideation, requirements, etc.) it does not.

6. The Features You Will Test (screen by screen)

This is your reference map of the whole application. Each entry says what the screen is for and what to check. The left sidebar groups the screens; we follow that order.

6.1 Signing in and onboarding

- **Sign up** (create workspace) — fields Full name, Work email, Password, Workspace/org name. Check validation (bad email, weak/blank password, duplicate email is rejected), the success state, and that **Continue** leads to the Setup checklist.
- **Log in** — Work email + Password. Check wrong-password handling and the account-locked state after several failed tries (you should see a clear message, not a raw error).
- **Forgot password / Reset** — request a reset, the "check your email" state, setting a new password, and the "invalid/expired link" state.
- **Accept invite** — if a teammate invites you, the link opens a join screen with your email pre-filled; you set a name and password to join their workspace. Test the expired-link path too.
- **Setup checklist** — the four guided steps (create project, connect machine, invite team, connect integrations). Check each link goes to the right place and the checklist reflects what you have done.

6.2 Setting up a project

- **Create a project** — name it, choose **greenfield** or **brownfield**, and **single** or **multi**-repo. Greenfield sends you to the Project Profile; brownfield sends you to the Onboarding hub. Test all four combinations.

- **Project Profile** — the important context that guides the AI (what the project is, constraints, etc.). Check editing, saving, and that required context is enforced: **Description is required** — Save with it empty should say why, put the cursor there, and keep everything else you typed.
- **Integrations** — the five capabilities you connect (doc store, task tracker, design source, code host, test management). Each shows Connected / Not connected / Failed. Code intelligence has no card: it is set up automatically. See 6.3.
- **Repositories** — add and assign the code repo(s) for the project, mark a default with **Set default** (the default marker should move to that row), and (multi-repo) pin work items to a repo.
- **Connect Machine** — covered in setup (5.7); also test the **Disconnect** flow and the offline state.

6.3 Connecting real tools (Integrations)

Integrations are organised by **capability**; each capability binds one vendor. All five below are wired to real APIs, so a wrong token should **fail the connection test** with a clear message. (Code intelligence is built in and set up automatically for every workspace, so it is not on this screen; an old link to it should return to the grid.)

Capability	Vendor	What you enter	What to test
Documentation store	Confluence	Space key, base URL, email + API token	Connect, edit, disconnect; a bad token fails the test; secrets are never shown back.
Task tracker	Jira	Project key, base URL, email + API token	Same as above; used later for publishing issues.
Design source	Figma	File key + access token	Connect + test; read-only use.
Code host	GitHub	Repo (optional) + access token (repo scope)	Connect; used by build phases to open PRs.
Test management	Xray	Project key + client id + client secret	Connect + test; used to import test results.

> **Good things to try to break here**
>
> Enter a clearly wrong token and confirm the **Test** step fails with a helpful message (not a stack trace). Try connecting the same capability twice (should say “already connected”). Confirm that after saving, the secret is never displayed again — only an “is configured” indicator.

6.4 Work Items

- **Work Items board** — columns are **Backlog** (nothing has run yet) followed by the lifecycle phases; the Tasks view is Backlog, Implementation, Review. Each card has a status pill: Not started, Running, **Waiting for you** (only when you may decide the gate or answer the question — otherwise a grey “Waiting for review / for an answer”), Done, Failed. Test creating each type and opening a card.
- **Work Item detail** — the view changes by type: **Epic**, **Task / Test case**, and **Defect** each render differently. Check the phases list and the **trigger** action.
- **Task breakdown** — review and accept the AI-generated breakdown of a work item into smaller pieces.
- **Test cases** — view and manage the generated test cases for a work item.

6.5 Phases & Runs

- **Phases & Runs** — pick a work item, see the full phase catalog, and **trigger a phase**. If your companion is offline, this screen blocks triggering and shows an offline state (that is correct behaviour, not a bug).
- **Run console** — the live output of a running phase. Watch it stream, and check it reflects success, failure, and “waiting for a clarification” clearly.
- **Run transcripts** — the full history of a finished run, for later reading.

6.6 Clarifications and Gates (staying in control)

- **Clarifications** — questions the AI asks, grouped in three tiers (Tier 1 must-answer, Tier 2 suggested, Tier 3 inferred defaults). Answering unblocks the run. Check the count badge in the sidebar and the notification bell update.
- **Gates** — approve or reject a phase's output. There are Pending and History tabs. Only a user holding the gate's **role** (or an admin) can decide — others should be blocked with a clear message.
- **Gate criteria checklist** — some gates require you to tick criteria before you can approve. Some criteria are computed by the platform and cannot be forced. Confirm “Approve” stays disabled until required criteria are met.

6.7 Seeing everything (observability)

- **Workspace Dashboard** — the “needs me” rollup: gates awaiting you, clarifications, your runs, project health. It is **personal**: it counts only gates you may decide and questions you may answer (“N waiting on you”), each row names its work item by number and title, and a run row says how long it has been going. “Published this week” counts by publish date. Check it updates as you work.
- **Artifacts** — browse everything the phases produced; open one to preview the rendered content and its version.
- **Publish to system of record** — from an artifact, export it to a connected tool (e.g. Confluence). This is **user-triggered** and only works after the gate is approved and the integration is connected.
- **Traceability** — the links between requirements, artifacts, work items, and tests. Check the links are correct and clickable, and try the filters: Phase, Type, and Highlight work item (the rest of the map dims; onboarding documents are under “Not tied to a work item”).
- **Run transcripts**, **API Contracts & Drift**, and **Run recovery** (take over a run whose companion went offline mid-phase).

6.8 Brownfield onboarding

- **Onboarding hub** — for existing projects: connect sources, extract and index existing docs, and run a gap report.
- **Source documents** — review what was ingested.
- **Gap report** — see what is missing versus what a healthy project should have.

6.9 Admin (workspace admins only)

- **Users & invitations** — invite teammates and manage members.

- **Roles & memberships** — assign each person roles per project (this decides who can approve which gates).
- **Test reporting** — CI test-result reporting and tokens (admin only).
- Non-admins should be politely blocked from all of the above — confirm that.

7. The Phase Catalog (the 16 lifecycle steps)

Each phase is one specialist AI agent. This table is the current, complete catalog. Most phases are “planning” phases that produce a document and stop at a gate. A few are “build” phases that actually change code. One reads your code without changing it.

#	Phase (trigger)	What it produces	Reviewer role	Kind
1	Ideation (ideate)	The initial idea shaped into a starting point	Product owner	Planning
2	Requirements (requirements)	Structured requirements	Product owner	Planning
3	PRD (prd)	Product requirements document	Product owner	Planning
4	Architecture (architecture)	Technical architecture	Architect	Planning
5	UX Design (ux-design)	UX/design direction	Designer	Planning
6	Task Breakdown (task-breakdown)	Work broken into buildable tasks	Tech lead	Planning
7	Implementation (implement)	Actual code, via a branch and pull request	Tech lead	**Build**
8	Code Review (code-review)	Review of the code (runs *inside* the build)	Reviewer	Auto (in build)
9	Testing (test)	Test plan and test cases	QA lead	Planning
10	Test Automation (test-automation)	Automated test scripts + imported results	QA lead	**Build**
11	Security (security)	Threat model, SBOM, security review + findings	Security engineer	Scan (reads code)
12	CI/CD (cicd)	Pipeline configuration	DevOps	**Build**
13	Deployment (deploy)	Deployment step	DevOps	Planning
14	Onboarding (onboard)	Sets up an existing project	Product owner	Planning
15	Context Bridge (context-bridge)	Bridges existing context	Architect	Planning
16	Context Extractor (analyze)	Extracts intelligence from existing code	Tech lead	Planning

7.1 The three kinds of phase, in plain terms

- **Planning phase** — the AI thinks and writes a document. It never touches code. It produces a draft and stops at a human gate. Most phases are these.
- **Build phase** (Implementation, Test Automation, CI/CD) — the AI actually changes code: it creates a working copy, commits, pushes, and opens a **pull request**, which a human then reviews and merges. These **require** your companion to be started with a real repo and the --worktrees folder.

- **Security scan** (Security) — special: it reads your real source code in a **read-only** throwaway copy to scan for vulnerabilities, but never changes or pushes anything. If no repo is connected it falls back to a lighter, document-only scan.

> **Two things not to test**
>
> **Code Review** is not a button you press on its own — it runs automatically inside the build phase, so do not look for a standalone “code-review” trigger. And an old **Context Snapshot** phase has been retired — it will not appear anywhere; that is correct.

7.2 The order you would run them

For a new (greenfield) project the natural order follows the numbers: ideation -> requirements -> PRD -> architecture -> UX -> task breakdown -> implementation -> testing -> test automation -> security -> CI/CD -> deployment. For an existing (brownfield) project you first run the onboarding/context phases. The platform does not force a rigid sequence — you trigger each phase on a work item when you are ready — but this order is the intended flow to test.

8. The Companion, in Detail

The companion is the one piece that runs on your laptop, and it is where build/security testing succeeds or fails. This section explains its commands so you can set it up right for each kind of testing.

8.1 What it is and why it exists

The companion (also called the “runner”) is a small program that quietly waits for the platform to hand it work, runs that work on your machine using your Claude login, and reports the result back. It is why phases can run without the platform paying a shared AI bill — the work happens on your computer, on your subscription.

8.2 The three commands

You run it as python -m runner.companion <command>. There are three commands, but for normal testing you mostly use the first two:

Command	What it does	When you use it
enroll	Registers your machine with the platform and prints a one-time token.	Usually done *for* you by the **Connect Machine** screen; you rarely type it by hand.
serve	The main worker. Waits for phases/builds and runs them. Keep it running while you test.	Every testing session, after setup.
run	Runs a single build task from a file, one-off.	Advanced/manual; not needed for normal QA.

8.3 The serve options that matter

The **Connect Machine** screen builds most of this for you, but here is what each option means so you can adjust it:

Option	Meaning
--api-url	Where the platform is — for you, http://localhost:8000 (your local backend).
--tenant	Your workspace id (the app fills this in).
--token	The one-time enrollment token from the Connect Machine screen.
--worktrees	A folder where the companion makes temporary working copies of repos. **Required for build and security phases.**
--repo-path	**Single-repo shorthand:** the one local clone that builds run against.
--repo NAME=PATH	**Multi-repo:** maps each repo (e.g. acme/api=C:\repos\api) to its local clone. Repeat once per repo. An explicit map always wins over --repo-path.
--headed	**Optional, for watching a test run:** shows the browser on screen during the Test Automation phase (09b) instead of running it invisibly, so you can watch the AI drive the app. Leave it off for normal testing.
--poll-interval / --heartbeat-interval	How often it checks for work / signals it is alive. Defaults are fine; leave them.

8.4 Which setup for which testing

- **Planning phases only** (ideation, requirements, PRD, architecture, UX, task breakdown, testing, deployment): the companion needs **no repo** — just serve with api-url, tenant, token. It reasons over the platform's stored context.
- **Build phases** (implementation, test automation, CI/CD): you **must** start serve with --worktrees **and** a repo (either --repo-path for single-repo, or one or more --repo NAME=PATH for multi-repo). Without them the build fails loudly — that is expected, not a bug.
- **Security phase**: give it --worktrees and the repo so it can scan real code. Without them it still runs but only does a lighter document-only scan.

> **Multi-repo safety**
> In a multi-repo project the companion will **never** guess which clone to use — if a repo isn't mapped, the build fails on purpose rather than pushing to the wrong GitHub repo. If you see that failure, add the missing --repo NAME=PATH and restart serve.

9. End-to-End User Journeys (walk-throughs)

These are the full journeys to test, written as a real user would perform them, click by click. Journey A is also your **smoke test** — do it first to prove your setup works, then explore the others.

9.1 Journey A — Greenfield, single repo (the golden path / smoke test)

Goal: start from nothing and drive a brand-new project through the early lifecycle, proving sign-up, project setup, companion, triggering, clarifications, and gates all work.

1. **Sign in.** App running at http://localhost:5173, you are logged into your own workspace (Section 5).
2. **Create a project.** Sidebar -> Work Items area, or Setup checklist -> **Create your first project**. Name it (e.g. “SpendLog”), choose **Greenfield** and **Single repo**. It takes you to the **Project Profile**.
3. **Fill the Project Profile.** Add the important context (what the project is about) and save.
4. **Confirm the companion is Ready.** The top-bar pill should say **Companion: Ready**. If not, start serve (Section 8).
5. **Create a work item.** Go to **Work Items**, create a **Story** (e.g. “Track daily spending”).
6. **Trigger the first phase.** Open the item (or go to **Phases & Runs**), pick the item, and trigger **Ideation**. Watch the **Run console** stream.
7. **Answer clarifications if asked.** If a badge appears on **Clarifications**, open it, answer the Tier-1 questions, and the run resumes.
8. **Approve the gate.** When the run finishes, a **gate** opens. Go to **Gates**, open the pending one, tick any required criteria, add a comment, and **Approve**. The work item advances.
9. **Repeat down the lifecycle.** Trigger **Requirements**, then **PRD**, then **Architecture**, approving each gate. Each produces an artifact you can open under **Artifacts**.
10. **Check traceability.** Open **Traceability** and confirm the artifacts link back to the work item and requirements.

> **Smoke-test pass criteria**
> If you reached an approved Architecture gate with artifacts visible and traceability linked — without an application error — your environment is healthy and you can test everything else with confidence.

9.2 Journey B — A build phase (code actually changes)

Goal: prove the platform can make real code changes and open a pull request. Needs the companion started with a repo + --worktrees (Section 8), and a GitHub integration connected.

1. Ensure your serve is running with --worktrees and --repo-path pointing at a real clone.
2. Connect the **Code host (GitHub)** integration for the project (Integrations screen).
3. On a work item that has been through task breakdown, trigger **Implementation**.
4. Watch the run: the companion makes a working copy, writes code, commits, pushes, and opens a **pull request** on GitHub. **Code Review** runs automatically inside this build.
5. A **merge gate** opens for a human. Review and decide.
6. Verify on GitHub that the branch and PR were actually created.

> **Multi-repo variant**
> For a multi-repo project, start serve with one --repo NAME=PATH per repo, and make sure the work item is **pinned to a repo** (multi-repo items must be — an unpinned one is meant to be confirmed at the Task Breakdown gate). Confirm the change lands in the *correct* repo.

9.3 Journey C — The security scan (read-only)

Goal: prove the security phase scans real code safely. Needs the companion with --worktrees and the repo.

1. On a work item tied to a repo, trigger **Security** from Phases & Runs.
2. The companion makes a **read-only** throwaway copy and scans the code (it never commits or pushes).
3. It produces security findings (and a threat model / SBOM / security review on a full run). Critical/High findings open **Defect** work items automatically.
4. Review the findings, and check that any Defect children appear under the work item.

> **Known behaviour on security runs**
>
> A security run can be long and may stop early if the AI hits a usage limit — that is not an application bug. Defect items are only opened when a run finishes reporting; a partial run intentionally opens none. Re-trigger when you have usage headroom.

9.4 Journey D — Brownfield onboarding

Goal: bring an existing project's docs and code into the platform.

1. Create a project and choose **Brownfield**. It takes you to the **Onboarding hub**.
2. Connect the source tools (e.g. Confluence/Jira) and your repo(s).
3. Run the extraction/indexing, then open **Source documents** to see what was ingested and the **Gap report** for what is missing.
4. From there, continue into the normal lifecycle phases.

9.5 Journey E — Clarifications and offline takeover (cross-cutting)

These behaviours sit on top of any journey and are worth testing directly:

- **Clarification round-trip:** trigger a phase; if it asks, the **Clarifications** badge appears and the run idles (costing nothing). Answer -> it resumes.
- **Offline takeover:** stop the companion (Ctrl-C), trigger a phase, and wait. After a timeout the run is parked for **human takeover** (visible on **Run recovery**) rather than hanging. Restart the companion to resume.
- **Companion status honesty:** the **Companion** pill should flip between Ready and Offline as you start/stop serve.

10. Is It a Bug, or Is It Meant to Work That Way?

Read this before logging behaviour that merely looks wrong. These are deliberate design decisions — reporting them as defects slows everyone down. Anything *not* on this list that feels broken, however, is worth logging.

Behaviour you might see	Verdict
A run stops partway with a message about a usage or rate limit.	**By design.** The AI has usage limits. Re-trigger later. Only log it if the *message* is a raw/technical error instead of a plain explanation.

Behaviour you might see	Verdict
A gate can be approved even though findings exist (e.g. security).	**By design.** Gates are advisory human checkpoints; the human decides. Findings are tracked separately.
A security run that ended early opened no Defect items.	**By design.** Defects are only opened when a run finishes reporting.
Phases & Runs won't let you trigger and shows an offline state.	**By design.** Your companion isn't running/enrolled. Start serve.
A build phase fails because no repo or no --worktrees was provided.	**By design.** Build phases need a real repo and worktrees folder.
There is no standalone “Code Review” button.	**By design.** Code review runs inside the build phase.
A saved integration secret is never shown back to you.	**By design.** Secrets are write-only; only an “is configured” flag shows.
You cannot approve a gate and get a “role” error (403).	**By design.** You need that gate's role on the project (or be an admin). Assign roles under Admin -> Roles.
Nothing was pushed to Confluence/Jira after a gate approval.	**By design.** Publishing is a separate, user-triggered step (Artifacts -> Publish), and only if that integration is connected.
You see another workspace's data.	**NOT by design — log P1 immediately.** Tenant isolation must be absolute.

11. Testing in Parallel Without Stepping on Each Other

Two testers plus two developers all share the same cloud brain. A little discipline keeps your results clean and your bugs reproducible.

- **Use your own workspace.** Each tester signs up their own workspace (tenant). Your data is then fully separate from the other tester's and the developers'. Name it clearly (Section 5.6).
- **Keep your code in sync.** All of you run the app and companion from the same project code. Before a testing session, pull the latest (git pull) and restart your app and companion, so you are testing the current build. If in doubt about which version to test, ask the team for the “known-good” commit for the day.
- **Know that workers are shared.** Everyone's companion listens on the same shared work queue. A phase you trigger normally runs on *your* companion, but the system is shared, so if something behaves oddly, note the time and your workspace name so a developer can trace it.
- **Record your environment in every bug.** Because the environment is shared, always note your workspace name, the work item key (e.g. WI-14), the phase, and the run's time. This makes any bug reproducible.
- **Don't test destructive things on shared data.** Stick to your own workspace's projects and items.

> **If two people report the same odd run**
>
> Because runs can execute on different machines, the same symptom might have two different causes. Always keep your bug reports tied to *your* workspace and run — don't merge your report with another tester's until a developer confirms they are the same issue.

12. Troubleshooting (fix it yourself first)

Ninety percent of setup snags are one of these. Try the fix here before asking a developer — that is the whole point of having this guide.

Symptom	Most likely cause & fix
Browser shows nothing at localhost:5173.	The app didn't start. Make sure **Docker Desktop is running**, then re-run docker compose --profile app up --no-deps backend worker frontend and watch for errors in that terminal.
App loads but login/signup fails or errors.	Your .env shared values are wrong or missing (database/Temporal). Re-check them against what the dev team gave you, then restart the app.
“Socket is not connected” or database errors.	The shared cloud is unreachable from your machine (network) or Docker dropped. Confirm you can reach the shared host; restart Docker and the app.
Companion won't start; prints “CANNOT START”.	The claude CLI isn't logged in, or an API key is set. Run claude to log in, and make sure no ANTHROPIC_API_KEY is forced in your shell.
A phase never runs / stays pending.	Your companion isn't running or isn't enrolled for this project. Check the **Companion: Ready** pill, that serve is up in its terminal, and that you enrolled for this project.
A build or security phase fails immediately.	You started serve without --worktrees and/or a repo. Restart it with both (Section 8).
A live tool step (publish to Confluence/Jira) does nothing.	That integration isn't connected for *your* project, or the token is wrong. Check Integrations shows it **Connected** and re-test the credentials.
Can't approve a gate (403 / role error).	You don't hold that gate's role. As your own workspace admin, assign yourself the role under Admin -> Roles & memberships.
The frontend seems stuck on old code after a pull.	Restart the frontend service (stop and re-run the compose command). On Windows the auto-reload sometimes misses changes.

When to actually ask a developer

Ask if: a shared secret in your .env looks wrong and you can't verify it; the shared cloud (database/Temporal) is down for everyone; or you hit an application error that this section doesn't cover and that blocks all testing. Everything else, try the fix above first.

13. How to Report a Bug (the format that gets it fixed)

Your bug reports are the product of all this testing. A good report gets fixed fast; a vague one bounces back with questions. Keep your own bug sheet in the shared format below.

13.1 Keep your own bug sheet

Each tester keeps a spreadsheet (Excel) of the bugs they find. Use these columns, in this order — they match how the development team already tracks bugs, so your sheet drops straight into their workflow:

Column	What goes in it
S.No	A running number (1, 2, 3, ...). Never reuse a number.

Column	What goes in it
Bug / Issue	A short, specific title. “Approve button stays disabled after all criteria ticked” — not “gate broken”.
Description	Optional longer detail if the title isn't enough. Many teams leave this empty when the title is clear.
Feature	Which area/screen (e.g. Gates, Integrations, Phases & Runs, Sign up).
Type	UI issue, Functional bug, Content/wording, Performance, Crash/Error, etc.
Reference	A screenshot pasted into the cell (see 13.3), or a note of where to look.
Priority	P1 / P2 / P3 (see 13.2).
Root Cause	Leave blank — the developers fill this when they investigate.
Fix Implemented	Leave blank — the developers fill this when they fix it.
Status	Open (you), then Fixed / Won't fix (developers).

13.2 Priority — how bad is it?

Priority	Use it when...	Examples
P1 (Critical)	Blocks testing, loses data, or breaks a core promise of the platform.	Can't log in; a phase can never be triggered; you can see another workspace's data; the app crashes.
P2 (Major)	A feature is clearly wrong or broken, but you can still work around it.	A gate approves without enforcing a required criterion; a wrong error message; a broken link on a key screen.
P3 (Minor)	Cosmetic or low-impact polish.	Misaligned button, typo, inconsistent spacing, slightly confusing label.

13.3 Attaching screenshots (important gotcha)

Screenshots make a report ten times more useful. You can paste an image directly into a spreadsheet cell, but be aware of one trap the team has hit:

> **The in-cell image trap**
>
> If you paste images *inside* cells, do **not** edit and re-save that sheet with automated tools — some tools silently drop in-cell images on save. If you hand your sheet to a developer, tell them it has in-cell screenshots so they preserve them. A safe alternative: keep screenshots in a folder and reference the filename in the **Reference** column.

13.4 What makes a report actionable

- **Steps to reproduce** — numbered, from a known starting point (“From Dashboard, click X, then Y”).
- **What you expected** vs. **what actually happened** — state both.
- **Environment** — your workspace name, the work item key (e.g. WI-14), the phase, and roughly when it happened (because the cloud is shared).
- **Evidence** — a screenshot, and the exact error text if any (type it out; don't just say “it errored”).
- **One bug per row** — don't combine several issues into one line.

- **Check Section 10 first** — make sure it isn't intended behaviour.

13.5 A worked example

> **Good bug report**
>
> **S.No:** 7 **Bug:** “Approve stays disabled after all gate criteria are ticked” **Feature:** Gates **Type:** Functional **Priority:** P2
> **Steps:** 1) Workspace “QA-Priya”, item WI-14, run Requirements to a gate. 2) Open Gates -> the pending gate -> criteria checklist. 3) Tick all required criteria. **Expected:** Approve becomes clickable. **Actual:** Approve stays greyed out; had to refresh the page to enable it. **Evidence:** screenshot gate-approve-disabled.png; no console error.

That is the whole job: set up once, understand the map, walk the journeys, and log what you find in this format. Welcome to the team — happy testing.